Network Programming
(Day 1)

Yunmin Go

School of AICE

Handong Global University

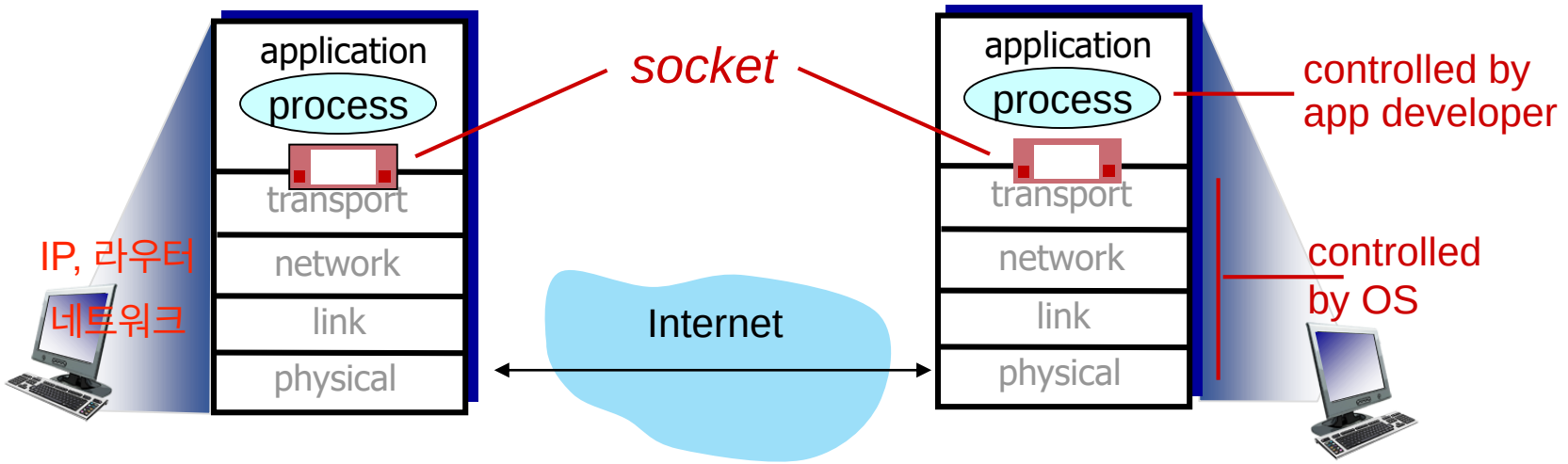
Network Programming (Day 1): roadmap

- Introduction
- TCP Socket Programming #1
- Internet Address and Domain Name
- TCP Socket Programming #2

Socket programming

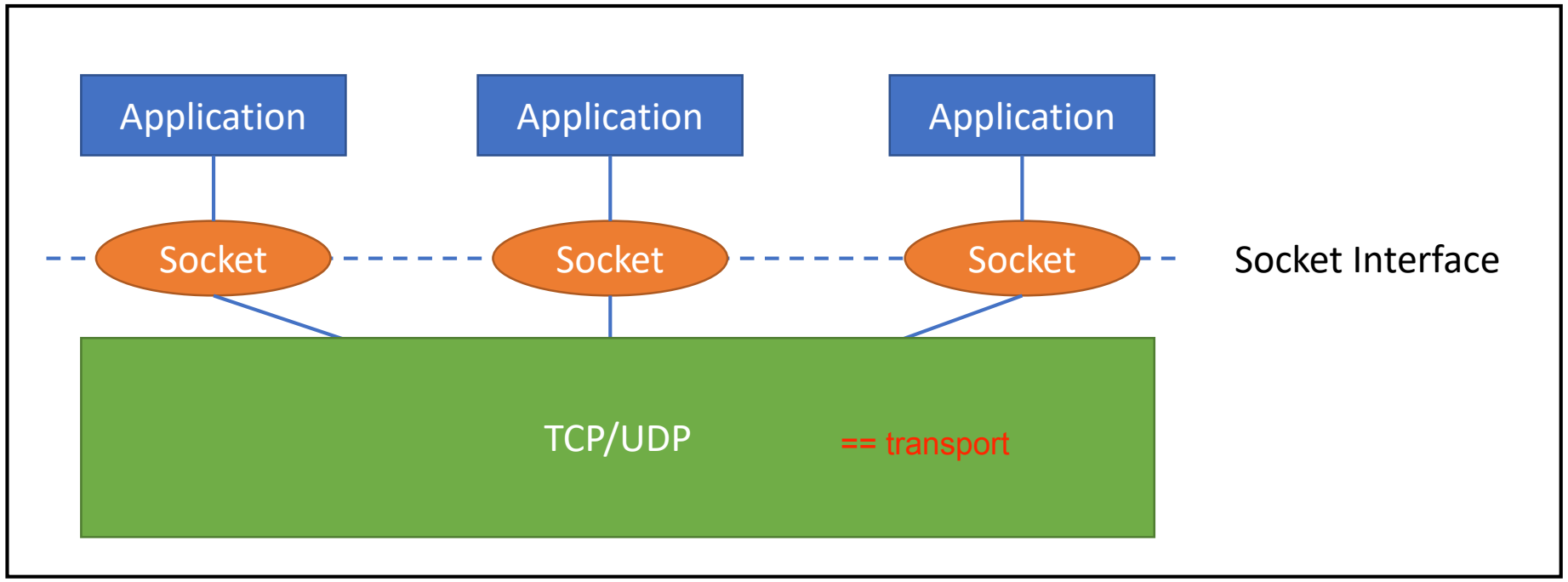
goal: learn how to build client/server applications that communicate using sockets

socket: door between application process and end-end-transport protocol



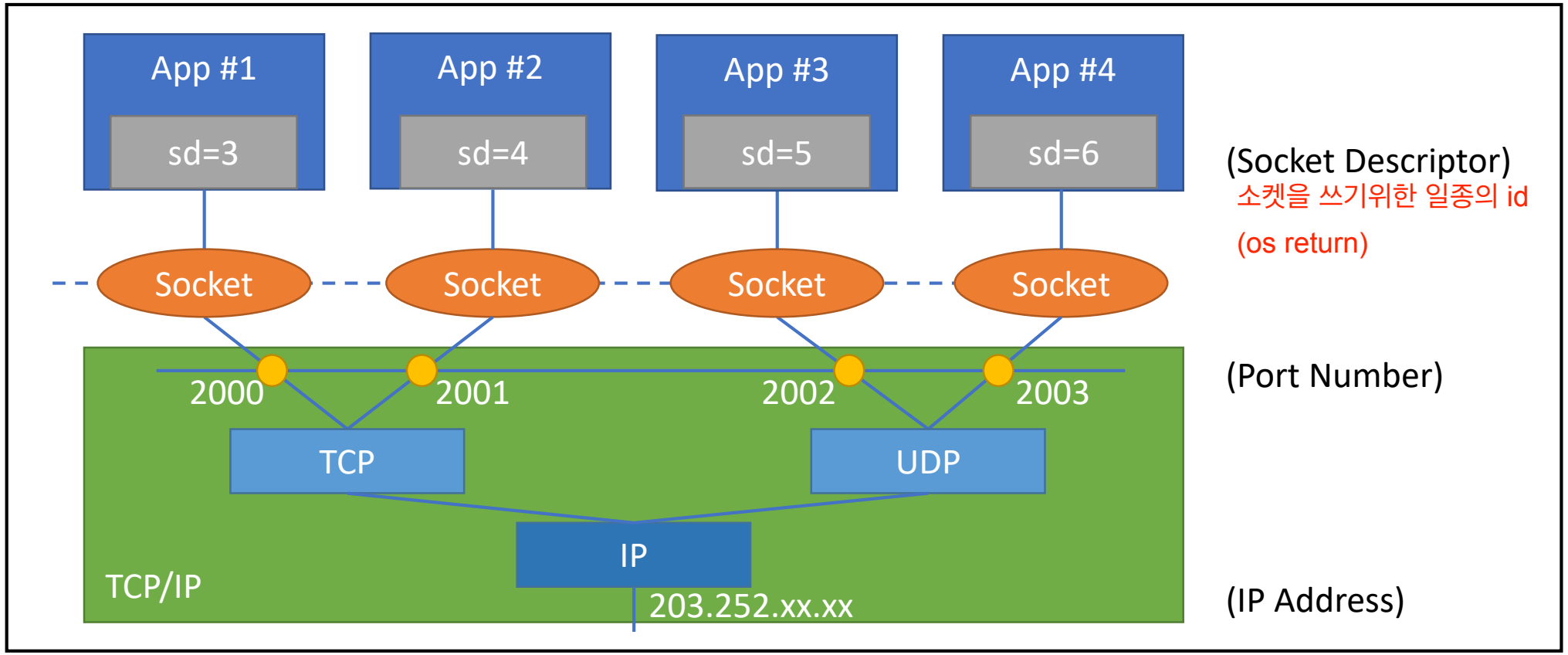
Socket

- Socket interface
  - Located between application and TCP, UDP



Socket

- Socket and TCP/UDP relationship



Socket

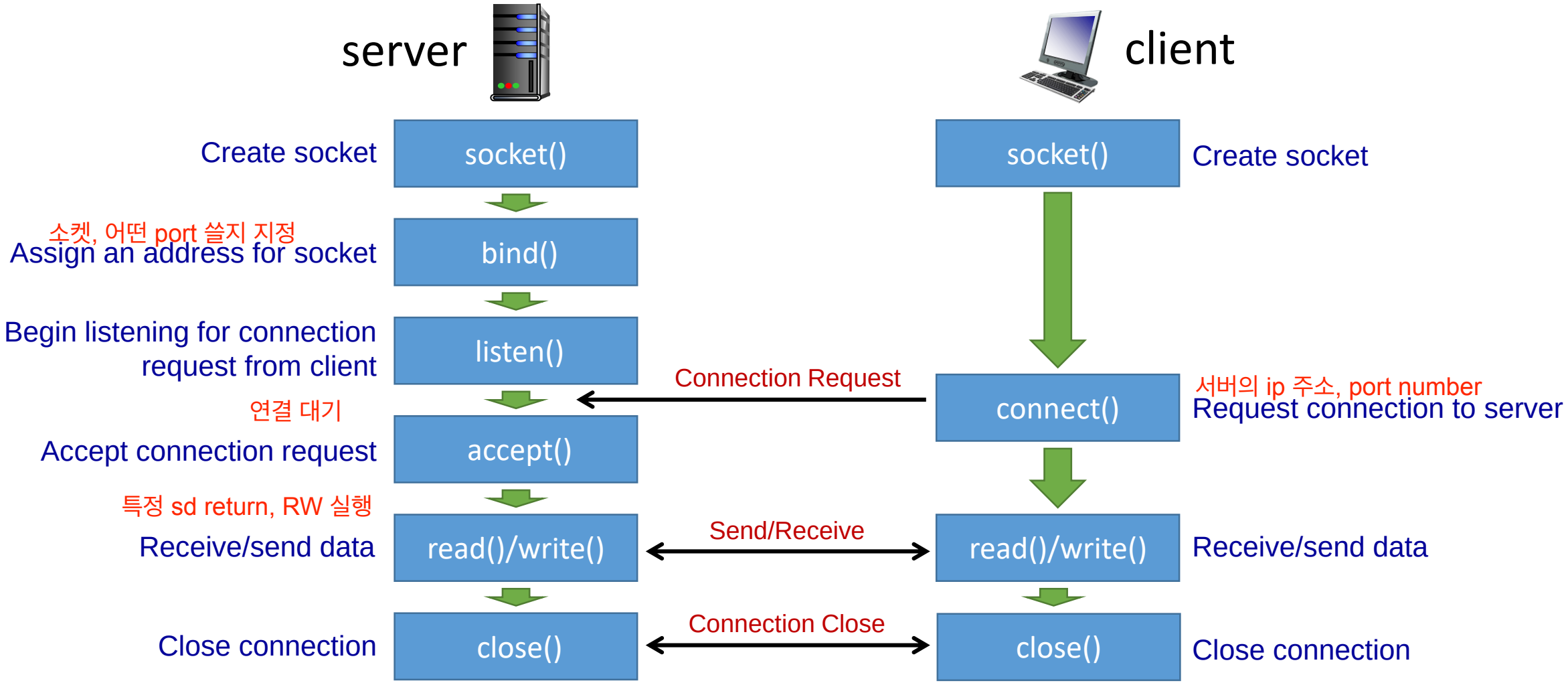
- Each socket is associated with five components
  - Protocol
    - Protocol family and protocol
  - Source address, source port 내 컴퓨터
  - Destination address, destination port 내가 연결하려는 destination

- Where to define components
  - Protocol: socket()
  - Source address and port: bind()
  - Destination address and port: connect(), sendto()

Network Programming (Day 1): roadmap

- Introduction
- TCP Socket Programming #1
- Internet Address and Domain Name
- TCP Socket Programming #2

TCP Server/Client Function Call



Example: tcp_server.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/types.h>
#include <sys/socket.h>

void error_handling(char *message);

int main(int argc, char *argv[])
{
   int serv_sock;
   int clnt_sock;

   struct sockaddr_in serv_addr;
   struct sockaddr_in clnt_addr;
   socklen_t clnt_addr_size;

   char message[] = "Hello World!";

   if (argc != 2){
      printf("Usage : %s <port>\n", argv[0]);
      exit(1);
   }
```

bind 함수에 주소정보 전달

```c
   serv_sock = socket(PF_INET, SOCK_STREAM, 0);
   if (serv_sock == -1)
      error_handling("socket() error");

   memset(&serv_addr, 0, sizeof(serv_addr));
   serv_addr.sin_family = AF_INET;
   serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
   serv_addr.sin_port = htons(atoi(argv[1]));

   if (bind(serv_sock, (struct sockaddr*) &serv_addr, sizeof(serv_addr)) == -1)
      error_handling("bind() error");

   if (listen(serv_sock, 5) == -1)
      error_handling("listen() error");

   clnt_addr_size = sizeof(clnt_addr);
   clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_addr, &clnt_addr_size);
   if (clnt_sock == -1)
      error_handling("accept() error");

   write(clnt_sock, message, sizeof(message));

   close(clnt_sock);
   close(serv_sock);
   return 0;
}
```

서버 소켓 (리스닝 소켓) 생성

socket()

주소정보(serv_addr) 초기화

bind()

주소정보 할당

listen()

client 대기

accept()

client 연결 허용

write()

sd 통해 데이터 송신

close()

socket 닫기

Example: tcp_client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/types.h>
#include <sys/socket.h>

void error_handling(char *message);

int main(int argc, char* argv[])
{
   int sock;
   struct sockaddr_in serv_addr;
   char message[30];
   int str_len = 0;
   int idx = 0, read_len = 0;

   if (argc != 3) {
      printf("Usage : %s <IP> <port>\n", argv[0]);
      exit(1);
   }
```

```
   sock = socket(PF_INET, SOCK_STREAM, 0);
   if (sock == -1)
      error_handling("socket() error");

   memset(&serv_addr, 0, sizeof(serv_addr));
   serv_addr.sin_family = AF_INET;
   serv_addr.sin_addr.s_addr = inet_addr(argv[1]);
   serv_addr.sin_port = htons(atoi(argv[2]));

   if (connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1)
      error_handling("connect() error!");

   while (read_len = read(sock, &message[idx++], 1))
   {
      if (read_len == -1)
      {
         error_handling("read() error!");
         break;
      }
      str_len += read_len;
   }
   printf("Message from server: %s \n", message);
   printf("Function read call count: %d \n", str_len);

   close(sock);
   return 0;
}
```

socket()
client 소켓 생성

server 에 연결 요청
connect()

read()
데이터 수신

close()

socket()

```
#include <sys/socket.h>
int socket(int domain, int type, int protocol)
```

Create an endpoint for communication

- *domain*: protocol family
  - PF_INET: IPv4 소켓이 사용할 프로토콜 체계 정보
  - PF_INET6: IPv6
- *type*: type of service 소켓 데이터 전송방식에 대한 정보
  - SOCK_STREAM: TCP
  - SOCK_DGRAM: UDP
  - SOCK_RAW: raw IP
- *protocol*: specifies the specific protocol
  - Usually 0 which means the default
  - IPPROTO_TCP
  - IPPROTO_UDP

두 컴퓨터간 통신에 사용되는 프로토콜 정보

- Return value
  - Success: a file descriptor for the new socket
  - Error: -1
- Example

```
sock = socket(PF_INET, SOCK_STREAM, 0);
if (sock == -1)
   error_handling("socket() error");
```

TCP Socket

```
sock = socket(PF_INET, SOCK_DGRAM, 0);
if (sock == -1)
   error_handling("socket() error");
```

UDP Socket

bind()

```
#include <sys/socket.h>
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

Assign an address to a socket 소켓에 인터넷 address 할당

- *sockfd*: file descriptor for the socket
- *addr*: address (IP address and port number) to assign a socket
- *addrlen*: the size (bytes) of the address structure pointed to by *addr*
  주소 구조체 변수의 길이(length)

- Return value
  - Success: 0
  - Error: -1
- Example

```
memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
serv_addr.sin_port = htons(atoi(argv[1]));

if (bind(serv_sock, (struct sockaddr*) &serv_addr, sizeof(serv_addr)) == -1)
   error_handling("bind() error");
```

listen()

```
#include <sys/socket.h>
int listen(int sockfd, int backlog);
```

Listen for connections on a socket 연결요청 대기상태로 진입

- Tell OS to receive and queue SYN packets
- TCP Server only
- *sockfd*: file descriptor for the socket (socket type should be SOCK_STREAM)
- backlog: the maximum number of connection requests that system can queue while it waits for the server to accept them 연결요청 대기 큐 길이

- Return value
  - Success: 0
  - Error: -1
- Example

```
if (listen(serv_sock, 5) == -1)
    error_handling("listen() error");
```

accept()

```
#include <sys/socket.h>
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

(소켓 descripter, 연결요청 할 client 의 주소, 주소변수 구조체 크기)

Accept a connection on a socket

- TCP Server only
- Block until a connection request arrives
- *sockfd*: file descriptor for the accepted socket (socket type should be SOCK_STREAM)
- *addr*: pointer to a sockaddr structure to be filled in with the address of the client socket
- *addrlen*: it will contain the actual size of the client address

- Return value
  - Success: file descriptor for the accepted sock (>0)
  - Error: -1
- Example

```
clnt_addr_size = sizeof(clnt_addr);
clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_addr, &clnt_addr_size);
if (clnt_sock == -1)
   error_handling("accept() error");
```

connect()

```
#include <sys/socket.h>
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

(소켓 discripter, 연결요청 할 client 의 주소, 주소변수 구조체 크기)

Initiate a connection on a socket

- For a TCP socket, it establishes a connection to the server
- For a UDP socket, it simply stores the server's address so that the client can use a socket description
- *sockfd*: file descriptor for the socket
- *addr*: address to connect
- *addrlen*: the size (bytes) of the address structure pointed to by *addr*

- Return value
  - Success: 0
  - Failure: -1
- Example

```
memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = inet_addr(argv[1]);
serv_addr.sin_port = htons(atoi(argv[2]));

if (connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1)
    error_handling("connect() error!");
```

read()

```
#include <sys/socket.h>
ssize_t read(int fd, void *buf, size_t count);
```

Read from a file descriptor

- Block until data received
- Attempts to read up to *count* bytes from file descriptor *fd* (socket) into the buffer starting at *buf*
- Return value
  - Success: the number of bytes read is returned, and the file position is advanced by this number
    - Zero indicates end of file = connection close
  - Error: -1

- Example

```
while (read_len = read(sock, &message[idx++], 1))
{
   if (read_len == -1)
   {
      error_handling("read() error!");
      break;
   }
   str_len += read_len;
}
```

write()

```
#include <sys/socket.h>
ssize_t write(int fd, const void *buf, size_t count);
```

Write to a file descriptor

- writes up to *count* bytes from the buffer starting at *buf* to the file (socket) referred to by the file descriptor *fd*
- Return value
  - Success: the number of bytes written is returned
  - Error: -1

- Example

```
char message[]="Hello World!";
write(clnt_sock, message, sizeof(message));
```

recv()

```
#include <sys/socket.h>
ssize_t recv(int socket, void *buffer, size_t length, int flags);
```

Receive a message from a connected socket

- Block until data received
- *socket*: socket file descriptor
- *buffer*: points to a buffer where the message should be stored
- *length*: the length in bytes of the buffer pointed to by the *buffer* argument (maximum length of the *buffer*)
- *flags*: type of message reception
  - 0 for regular data
- Return value
  - Success: the number of bytes received
    - Zero indicates the connection close
  - Error: -1
- Example

```
while((str_len = recv(recv_sock, buf, sizeof(buf), 0)) != 0)
{
   if (str_len == -1)
      continue;
   buf[str_len]=0;
    puts(buf);
}
```

send()

```
#include <sys/socket.h>
ssize_t send(int socket, const void *buffer, size_t length, int flags);
```

Send a message on a socket

- Transmit the data in *buffer* upto *length* bytes
- *socket*: socket file descriptor
- *buffer*: buffer containing the message to send
- *length*: the length of the message in bytes.
- *flags*: type of message transmission
  - 0 for regular data

- Return value
  - Success: the number of bytes transmitted
  - Error: -1
- Example

```
write(sock, "123", strlen("123"));
send(sock, "4", strlen("4"), MSG_OOB);
```

close()

```
#include <unistd.h>
int close(int fd);
```

Close a file descriptor

- Prevent any more read and writes to the socket
- If the remote side calls recv(), it will return 0
- If the remote side calls send(), it will receive a signal SIGPIPE and send() will return -1 and errno will be set to EPIPE.
- Return value
  - Success: 0
  - Error: -1
- Example

```
close(clnt_sock);
close(serv_sock);
```

Review: tcp_server.c & tcp_client.c

```
// Server
serv_sock = socket(PF_INET, SOCK_STREAM, 0);
if (serv_sock == -1)
   error_handling("socket() error");

memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
serv_addr.sin_port = htons(atoi(argv[1]));

if (bind(serv_sock, (struct sockaddr*) &serv_addr, sizeof(serv_addr)) == -1)
   error_handling("bind() error");

if (listen(serv_sock, 5) == -1)
   error_handling("listen() error");

clnt_addr_size = sizeof(clnt_addr);
clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_addr, &clnt_addr_size);
if (clnt_sock==-1)
   error_handling("accept() error");

write(clnt_sock, message, sizeof(message));

close(clnt_sock);
close(serv_sock);
```

socket()
서버 소켓 생성

서버 소켓의 주소정보(IP, PORT정보) 초기화

my address

bind()
및 할당

listen()
클라이언트 연결 대기

accept()
클라이언트 연결 허용

write()
sd통해 데이터 송신

close()

```
// Client
sock = socket(PF_INET, SOCK_STREAM, 0);
if (sock == -1)
   error_handling("socket() error");

memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = inet_addr(argv[1]);
serv_addr.sin_port = htons(atoi(argv[2]));

if (connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1)
   error_handling("connect() error!");

while (read_len = read(sock, &message[idx++], 1))
{
   if (read_len == -1)
   {
      error_handling("read() error!");
      break;
   }
   str_len += read_len;
}
printf("Message from server: %s \n", message);
printf("Function read call count: %d \n", str_len);

close(sock);
```

socket()
클라이언트 소켓 생성

서버 소켓 주소정보 초기화

connect()
클라이언트 소켓, connect 함수호출 통해 서버로 연결 요청

read()
eof t | 0 반환, 소켓 닫혔을때
서버로부터 데이터 수신

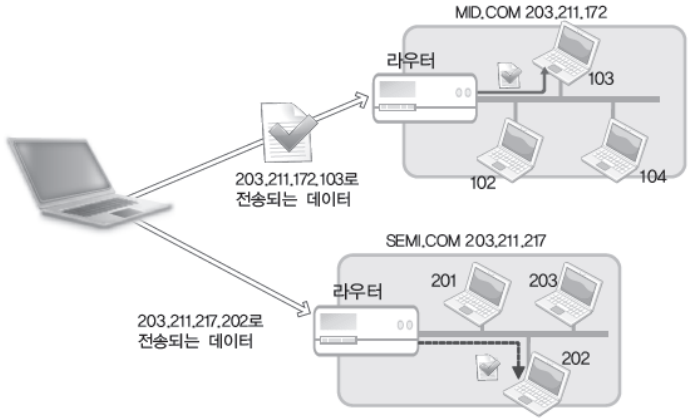
close()

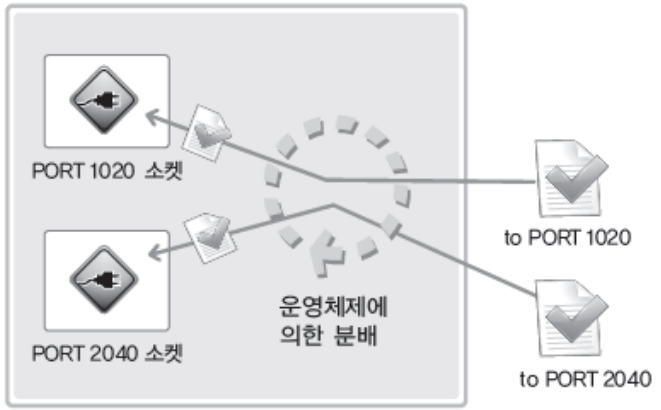
Network Programming (Day 1): roadmap

- Introduction
- TCP Socket Programming #1
- Internet Address and Domain Name
- TCP Socket Programming #2

Internet Address

- IP Address
  - Used to identify computers on the Internet
  - There are IPv4 (4-byte address) and IPv6 (16bytes address)
  - When creating a socket, we must specify a basic protocol
- Port
  - Used to identify sockets in the computer
  - Port number is represented in 16bits
    → Total range 0 ~ 65535
  - Well-known port: 0 ~ 1023
    - its use has already been decided
    - e.g., 22 for ssh, 80 for web, etc
  - Ephemeral port: a short-lived transport protocol port
    - allocated automatically from a predefined range by the IP stack software
    - e.g., Linux: 32768 to 60999





Address Structure

- Defined in <netinet/in.h>

```
struct sockaddr {
    u_short         sa_family;    // Address family
    char            sa_data[14];  // address
};
```

```
struct sockaddr_in {
    sa_family_t     sin_family;   // Address family: AF_INET
    uint16_t        sin_port;     // Port number (16bits), Network byte order
    struct in_addr  sin_addr;     // IP address (32bits), Network byte order
    char            sin_zero[8];  // Unused
};
```

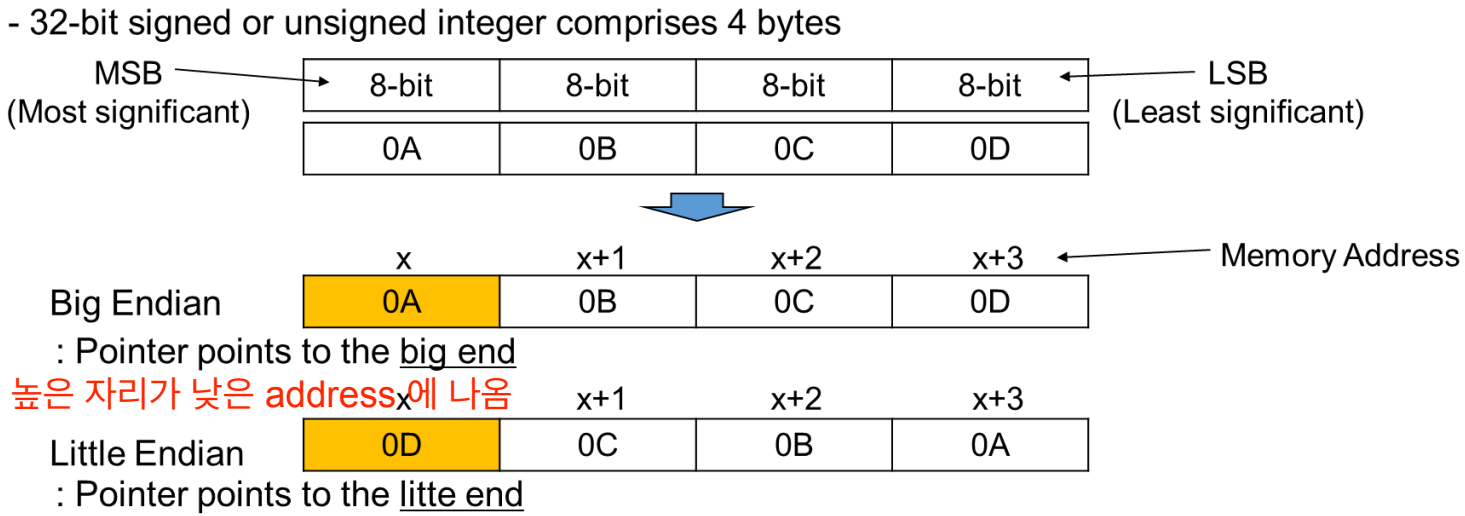
```
struct in_addr {
    in_addr_t       s_addr;       // IPv4 Internet address (32bits)
};
```

```
memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
serv_addr.sin_port = htons(atoi(argv[1]));

if (bind(serv_sock, (struct sockaddr*) &serv_addr, sizeof(serv_addr)) == -1)
    error_handling("bind() error");
```

※ INADDR_ANY: the socket will be bound to all local interfaces

Byte Order Conversion

- Byte ordering
  - Little endian: least significant byte first converting 필요
  - Big endian: most significant byte first
- Network byte order = big endian
  - Most-significant byte at least address of a word
  - Host byte order depends on host's CPU



host - network 간, 소켓 주소 구조체 변수에 값 채우기 위해서 --> Network Byte Order로 변환해서 저장 필요.

htons(), ntohs(), htonl(), ntohl()

```
#include <arpa/inet.h>
uint16_t htons(uint16_t hostshort); // convert unsigned short integer from host byte order to network byte order
uint16_t ntohs(uint16_t netshort);  // convert unsigned short integer from network byte order to host byte order
uint32_t htonl(uint32_t hostlong);  // convert unsigned integer from host byte to network byte order
uint32_t ntohl(uint32_t netlong);   // convert unsigned integer from network byte order to host byte order
```

hostshort to network short

nettshort to network short

Convert values between host and network byte order

- Example: endian_conv.c

```
unsigned short host_port = 0x1234;
unsigned short net_port;
unsigned long host_addr = 0x12345678;
unsigned long net_addr;

net_port = htons(host_port);
net_addr = htonl(host_addr);

printf("Host ordered port: %#x \n", host_port);
printf("Network ordered port: %#x \n", net_port);
printf("Host ordered address: %#lx \n", host_addr);
printf("Network ordered address: %#lx \n", net_addr);
```

```
>> Host ordered port: 0x1234
>> Network ordered port: 0x3412
>> Host ordered address: 0x12345678
>> Network ordered address: 0x78563412
```

inet_addr(), inet_aton ()

string address 로 받아 -> numeric 으로 return

```
#include <arpa/inet.h>
in_addr_t inet_addr(const char *string);
int inet_aton(const char *string, struct in_addr *addr)
```

IPv4 address manipulation

- Convert dotted decimal IP address (String) to 32bit big-endian integer
- inet_addr()
  - Return value
    - Success: Internet address (32bit Integer)
    - Error: -1
- inet_aton()
  - Similar with inet_addr(), but the result value is returned using argument
  - Return value
    - Success: 1 (true)
    - Error: 0 (false)

- Example (inet_addr.c)

```
char *addr1 = “1.2.3.4”;
char *addr2 = “127.232.124.79”;
struct sockaddr_in addr_inet;

unsigned long conv_addr = inet_addr(addr1);
if (conv_addr == INADDR_NONE)
   printf("Error occured! \n");
else
   printf("Network ordered integer addr#1: %#lx \n", conv_addr);

if(!inet_aton(addr, &addr_inet.sin_addr))
   printf("Conversion error\n”);
else
   printf(" Network ordered integer addr#2: %#x \n“, addr_inet.sin_addr.s_addr);

>> Network ordered integer addr: 0x4030201
>> Network ordered integer addr: 0x4f7ce87f
```

inet_ntoa()

numeric 숫자 -> string ip address

```
#include <arpa/inet.h>
char *inet_ntoa(struct in_addr adr);
```

IPv4 address manipulation

- Convert 32bit big-endian integer to dotted decimal IP address (String)
- inet_ntoa()
  - Convert string IP address to dotted decimal IP address
  - Return value
    - Success: converted dotted decimal IP address (String)
    - Error: -1
  - Return value

- Example: inet_ntoa.c

```
struct sockaddr_in addr1, addr2;
char *str_ptr;
char str_arr[20];

addr1.sin_addr.s_addr = htonl(0x1020304);
addr2.sin_addr.s_addr = htonl(0x1010101);

str_ptr = inet_ntoa(addr1.sin_addr);
strcpy(str_arr, str_ptr);
printf("Dotted-Decimal notation1: %s \n", str_ptr);

>> Dotted-Decimal notation1: 1.2.3.4
```

gethostbyname()

DNS (Domain Name System)

```
#include <netdb.h>
struct hostent *gethostbyname(const char *hostname);
```

Get host information by taking host name

- We can obtain the host's official name, aliases, and IP addresses by *hostname*

Text

- *hostname*: a hostname or an IPv4 address in standard dot notation
- Return value
  - Success: structure type of *hostent* for given host name
  - Error: NULL

- Example: gethostbyname.c

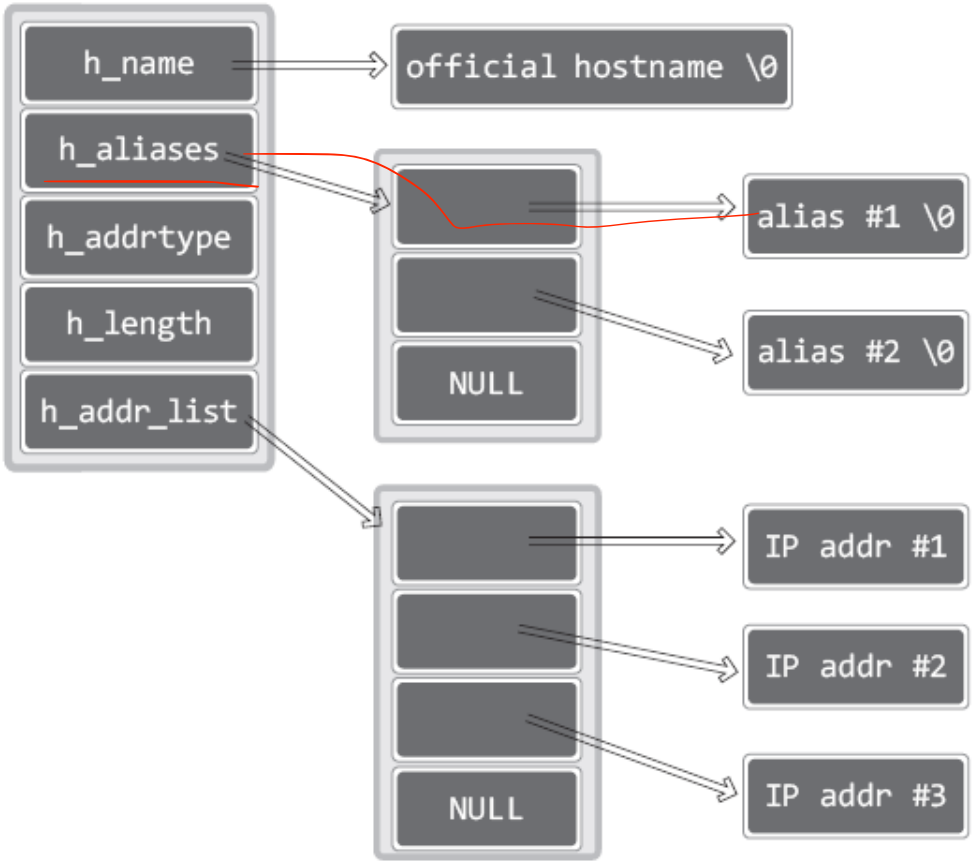
```
struct hostent *host;
…
host = gethostbyname(argv[1]);
printf("Official name: %s \n", host->h_name);

for (i = 0; host->h_aliases[i]; i++)
   printf("Aliases %d: %s \n", i+1, host->h_aliases[i]);

printf("Address type: %s \n",
         (host->h_addrtype==AF_INET)?"AF_INET":"AF_INET6");

for (i = 0; host->h_addr_list[i]; i++)
   printf("IP addr %d: %s \n", i+1,
           inet_ntoa(*(struct in_addr*)host->h_addr_list[i]));
```

struct hostent

- Defined in netdb.h

```
#define  h_addr haddr_list[0]
struct hostent {
    char    *h_name;        // official name of host
    char    **h_alias;      // alias list
    int     h_addrtype;     // host address type
    int     h_length;       // length of address
    char    **h_addr_list;  // list of addresses
};
```

여러 개의 ip 주소를 사용할 수 있기 때문에 list 형태로 사용
hostend 구조를 통해 탐색



gethostbyaddr()

```
#include <netdb.h>
struct hostent *gethostbyaddr(const char *addr, socklen_t len, int family);
```

Get host information by taking network byte order address

- We can obtain host's official name, aliases, and IP addresses by *addr* (network byte or address)
- *addr*: address of host (type = struct in_addr)
- len: length of *addr* (IPv4=4, IPv6=16)
- *familiy*: address family (AF_INET=IPv4, AF_INET6=IPv6)
- Return value
  - Success: structure type of *hostent* for given host name
  - Error: NULL

- Example (gethostbyaddr.c)

```
struct hostent *host;
addr.sin_addr.s_addr = inet_addr(argv[1]);
host = gethostbyaddr((char*)&addr.sin_addr, 4, AF_INET);
printf("Official name: %s \n", host->h_name);

for (i = 0; host->h_aliases[i]; i++)
   printf("Aliases %d: %s \n", i+1, host->h_aliases[i]);

printf("Address type: %s \n",
         (host->h_addrtype==AF_INET)?"AF_INET":"AF_INET6");

for (i = 0; host->h_addr_list[i]; i++)
   printf("IP addr %d: %s \n", i+1,
            inet_ntoa(*(struct in_addr*)host->h_addr_list[i]));
```

Network Programming (Day 1): roadmap

- Introduction
- TCP Socket Programming #1
- Internet Address and Domain Name
- TCP Socket Programming #2

TCP: overview RFCs: 793,1122, 2018, 5681, 7323

- point-to-point:
  - one sender, one receiver
- reliable, in-order *byte stream:* 데이터 수신 여부, 확인절차 거침, 신뢰 가능
  - no “message boundaries"
- full duplex data: 이중
  - bi-directional data flow in same connection
  - MSS: maximum segment size
- cumulative ACKs
- pipelining:
  - TCP congestion and flow control set window size 혼잡
- connection-oriented:
  - handshaking (exchange of control messages) initializes sender, receiver state before data exchange
- flow controlled:
  - sender will not overwhelm receiver

No Message boundary in TCP

- A "message boundary" is the separation between two messages being sent over a protocol.
- UDP preserves message boundaries
  - E.g., if the server calls write() twice, the client needs to call read() twice.
- TCP does not preserve message boundaries
  - E.g., even if the server calls write() twice, the client can read all data at once.

```
// Server
char message1[] = “Hello World!111”;
char message2[] = “Hello World!222”;
…
write(clnt_sock, message1, sizeof(message1));
write(clnt_sock, message2, sizeof(message2));
```

```
// Client
sleep(1);
read_len = read(sock, message, sizeof(message));
printf("Message from server: ");
for (i = 0; i < sizeof(message); i++)
   printf(“%c”, message[i]);
printf(“Read_len: %d \n", read_len );
```

Results

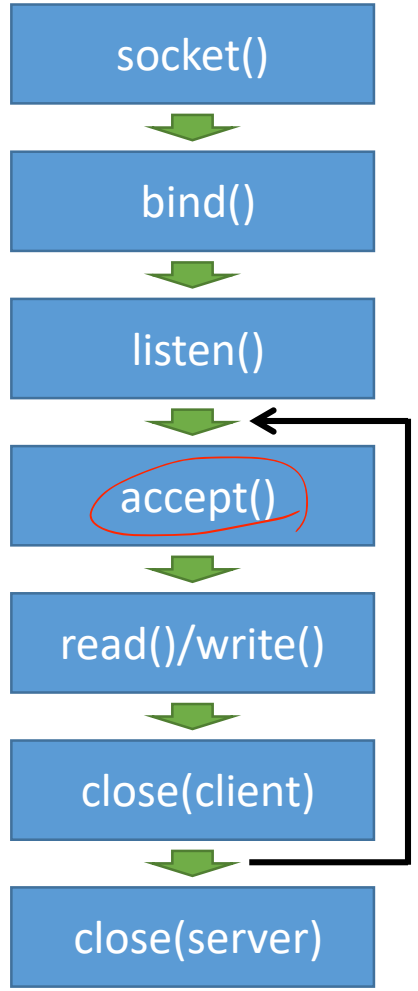
```
yunmin@ym-ubnutu:~/Workspace$ ./hclient2 127.0.0.1 9191
Message from server: Hello World!111Hello World!222◆◆p^◆
Read_len: 32
```

Server Type

- Iterative server
  - A single server process receives and handles incoming requests on a “well-known” port
  - Most UDP servers are iterative

- Concurrent server
  - A separate process to handle each client
  - The main server process creates a new service process to handle each client
  - Most TCP servers are concurrent: for ease of connection management

accept 함수 반복호출,
계속해서 들어오는 클라이언트의 연결요청 수락



Iterative vs. Concurrent servers

Iterative server skeleton

```
int sockfd, newsockfd;
if ((sockfd = socket(…)) < 0)
   err_sys(“socket error”);
if ((bind(sockfd, …) < 0)
   err_sys(“bind error”);
if (listen(sockfd, 5))
   err_sys(“listen error”);
for (;;) {
   newsockfd = accept(sockfd, …);
   if (newsockfd < 0)
      err_sys(“accept error”);
   doit(newsockfd);
   close(newsockfd);    한번의 요청만 accept 후 close (종료)
}
```

Concurrent server skeleton

```
int sockfd, newsockfd;
if ((sockfd = socket(…)) < 0)
   err_sys(“socket error”);
if ((bind(sockfd, …) < 0)
   err_sys(“bind error”);
if (listen(sockfd, 5))
   err_sys(“listen error”);
for (;;) {
   newsockfd = accept(sockfd, …);
   if (newsockfd < 0)
      err_sys(“accept error”);
   if (fork() == 0) {
      close(sockfd);
      doit(newsockfd);
      exit(0);
   }
   else {
      close(newsockfd);
   }
}
```

Example: echo_server.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define BUF_SIZE 1024
void error_handling(char *message);

int main(int argc, char *argv[])
{
   int serv_sock, clnt_sock;
   char message[BUF_SIZE];
   int str_len, i;

   struct sockaddr_in serv_adr;
   struct sockaddr_in clnt_adr;
   socklen_t clnt_adr_sz;

   if (argc != 2) {
      printf("Usage : %s <port>\n", argv[0]);
      exit(1);
   }

   serv_sock=socket(PF_INET, SOCK_STREAM, 0);
   if (serv_sock == -1)
      error_handling("socket() error");

   memset(&serv_adr, 0, sizeof(serv_adr));
   serv_adr.sin_family = AF_INET;
   serv_adr.sin_addr.s_addr = htonl(INADDR_ANY);
   serv_adr.sin_port = htons(atoi(argv[1]));

   if (bind(serv_sock, (struct sockaddr*)&serv_adr, sizeof(serv_adr)) == -1)
      error_handling("bind() error");

   if (listen(serv_sock, 5) == -1)
      error_handling("listen() error");

   clnt_adr_sz=sizeof(clnt_adr);
   for (i = 0; i < 5; i++)
   {
      clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_adr, &clnt_adr_sz);
      if (clnt_sock == -1)
         error_handling("accept() error");
      else
         printf("Connected client %d \n", i+1);

      while ((str_len = read(clnt_sock, message, BUF_SIZE)) != 0)
         write(clnt_sock, message, str_len);

      close(clnt_sock);
   }

   close(serv_sock);
   return 0;
}
```

5개의 클라이언트에게, 순서대로 echo 서비스 제공

read 한 문자열,
그대로 write (ehco 함)

Example: echo_client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define BUF_SIZE 1024
void error_handling(char *message);

int main(int argc, char *argv[])
{
   int sock;
   char message[BUF_SIZE];
   int str_len;
   struct sockaddr_in serv_adr;

   if (argc != 3) {
      printf("Usage : %s <IP> <port>\n", argv[0]);
      exit(1);
   }

   sock=socket(PF_INET, SOCK_STREAM, 0);
   if (sock == -1)
      error_handling("socket() error");

   memset(&serv_adr, 0, sizeof(serv_adr));
   serv_adr.sin_family = AF_INET;
   serv_adr.sin_addr.s_addr = inet_addr(argv[1]);
   serv_adr.sin_port = htons(atoi(argv[2]));
```

```
   if (connect(sock, (struct sockaddr*)&serv_adr, sizeof(serv_adr)) == -1)
      error_handling("connect() error!");
   else
      puts("Connected.....");

   while(1)
   {
      fputs("Input message(Q to quit): ", stdout);
      fgets(message, BUF_SIZE, stdin);

      if (!strcmp(message,"q\n") || !strcmp(message,"Q\n"))
         break;

      str_len = write(sock, message, strlen(message));

      recv_len = 0;
      while (recv_len < str_len)
      {
         recv_cnt = read(sock, &message[recv_len], BUF_SIZE-1);
         if (recv_cnt == -1)
            error_handling("read() error!");
         recv_len+=recv_cnt;
      }
      message[recv_len]=0;
      printf("Message from server: %s", message);
   }
   close(sock);
   return 0;
}
```

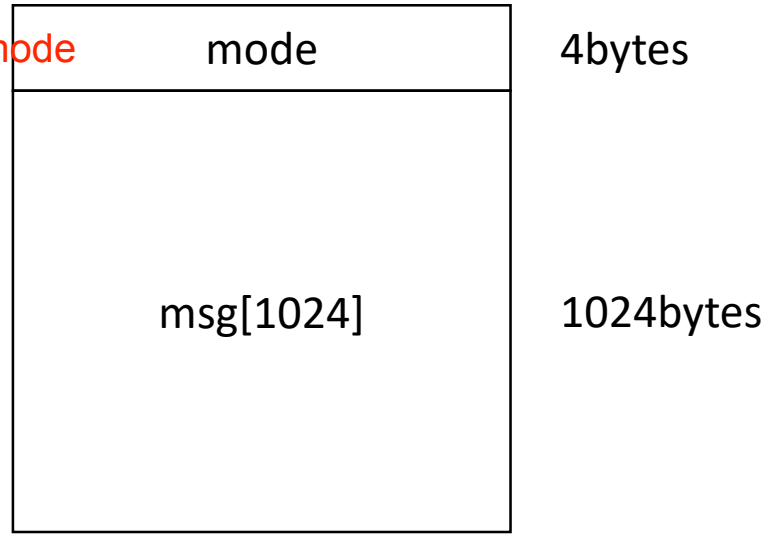
struct를 이용한 패킷 설계

- struct를 이용한 패킷 설계

단순히 char 배열로 보내는것 x,
원하는 field 만 define

```
#define BUF_SIZE 1024
…
typedef struct {
    unsigned int mode;
    unsigned char msg[BUF_SIZE];
} pkt_t;
```

데이터 처리 mode



struct를 이용한 패킷 설계

- 예제: client

```
...
send_pkt = (pkt_t *) malloc(sizeof(pkt_t));
recv_pkt = (pkt_t *) malloc(sizeof(pkt_t));
printf("sizeof(pkt_t)=%d\n", sizeof(pkt_t));
while (1)
{
   // Mode, Message
   memset(send_pkt, 0, sizeof(pkt_t));
   memset(recv_pkt, 0, sizeof(pkt_t));

   printf("Select a mode (0~1, Q to quit): ");
   scanf("%s", str);

   if (!strcmp(str,"q") || !strcmp(str,"Q"))
      break;
   else if (!isdigit(str[0]))
      mode = -1;
   else
      mode = atoi(str);

   if (mode < 0 || mode > 1)
   {
      printf("Undefined mode!\n");
      continue;
   }
      printf("Input message: ");
      scanf("\n%[^\n]s", str);

      send_pkt->mode = mode;
      strcpy(send_pkt->msg, str);

      // Send a pkt to server
      write(sock, send_pkt, sizeof(pkt_t));

      // Receive a pkt from server
      read(sock, recv_pkt, sizeof(pkt_t));

      printf("Message from server: %s \n", recv_pkt->msg);
}
```

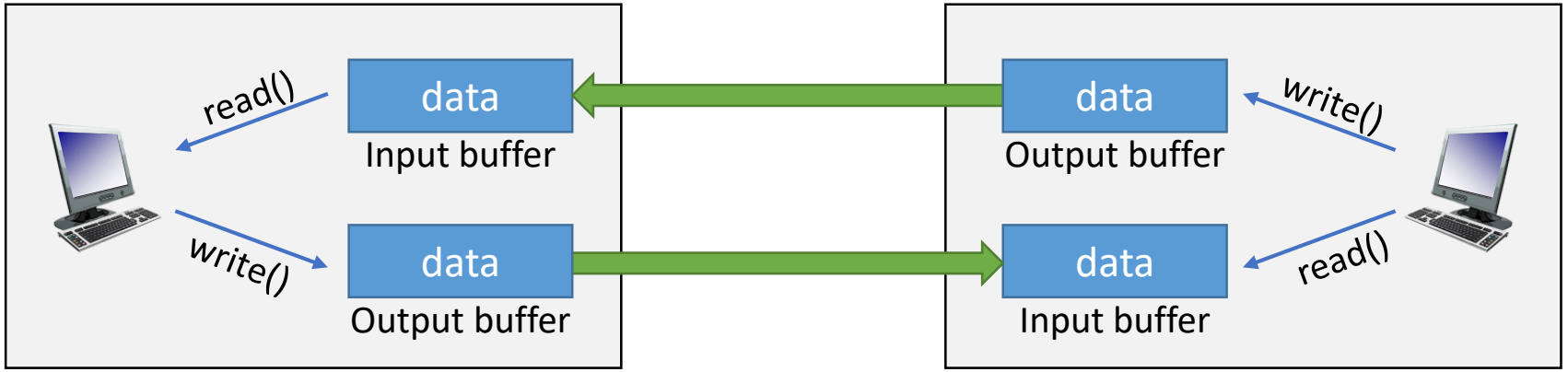
struct를 이용한 패킷 설계

- 예제: server

```
...
recv_pkt = (pkt_t *) malloc(sizeof(pkt_t));
while ((str_len = read(clnt_sock, recv_pkt, sizeof(pkt_t))) != 0)
{
   send_pkt = (pkt_t *) malloc(sizeof(pkt_t));
   send_pkt->mode = recv_pkt->mode;

   // String conversion
   switch (recv_pkt->mode)
   {
      case 0:
         strcpy(send_pkt->msg, recv_pkt->msg);
         break;
      case 1:
         strcpy(send_pkt->msg, recv_pkt->msg);
         strupr(send_pkt->msg);
         break;
      default:
         printf("Undefined mode! \n");
         break;
   }
   printf("Mode %d: %s -> %s \n", recv_pkt->mode, recv_pkt->msg, send_pkt->msg);

   // Send msg to client
   write(clnt_sock, send_pkt, sizeof(pkt_t));
}
```

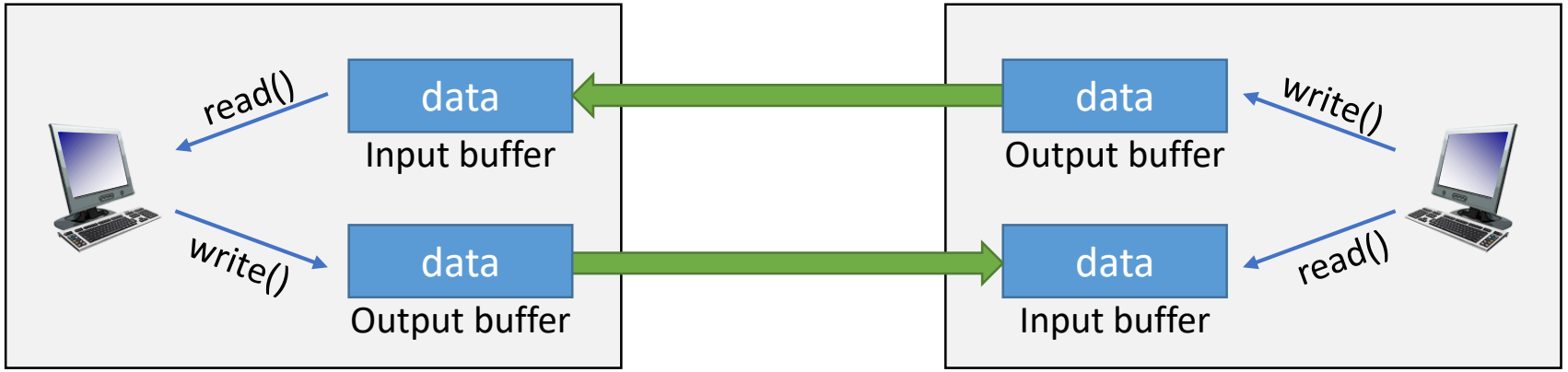
Input & Output Buffer for TCP Socket

- Input buffer and output buffer exists on the TCP socket
- Input buffer and output buffer are created when socket is created
- Even if the socket is closed, data remaining in the output buffer will continue to be transmitted
- When the socket is closed, the data remaining in the input buffer is destroyed



Half-close

- Meaning of close()
  - It means complete destruction of the socket
  - No more I/O is possible
  - If the data transmitting and receiving have not been completed yet, it causes problems
- Half-close
  - Close only one of the input or output streams



shutdown()

```
#include <unistd.h>
int shutdown(int socket, int how);
```

Shut down socket send and receive operations

- This function shall cause all or part of a full-duplex connection on the socket
- *socket*: socket file descriptor
- *how*: type of shutdown
  - SHUT_RD (0): disables further receive operations
  - SHUT_WR (1): disables further send operations
  - SHUT_RDWR (2): disables further send and receive operations
- The shutdown() does not actually free up the socket descriptor. To free the descriptor, use close()

- Return value
  - Success: 0
  - Error: -1
- Example

```
shutdown(clnt_sd, SHUT_WR);
read(clnt_sd, buf, BUF_SIZE);
printf("Message from client: %s \n", buf);

close(clnt_sd); close(serv_sd);
```

Example: file_server.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define BUF_SIZE 30
void error_handling(char *message);

int main(int argc, char *argv[])
{
   int serv_sd, clnt_sd;
   FILE * fp;
   char buf[BUF_SIZE];
   int read_cnt;

   struct sockaddr_in serv_adr, clnt_adr;
   socklen_t clnt_adr_sz;

   if (argc != 2) {
      printf("Usage: %s <port>\n", argv[0]);
      exit(1);
   }

   fp = fopen("file_server.c", "rb");
   serv_sd = socket(PF_INET, SOCK_STREAM, 0);
```

file의 데이터 크기에 따라 닫을지 안닫을지

```c
   memset(&serv_adr, 0, sizeof(serv_adr));
   serv_adr.sin_family = AF_INET;
   serv_adr.sin_addr.s_addr = htonl(INADDR_ANY);
   serv_adr.sin_port = htons(atoi(argv[1]));

   bind(serv_sd, (struct sockaddr*)&serv_adr, sizeof(serv_adr));
   listen(serv_sd, 5);

   clnt_adr_sz = sizeof(clnt_adr);
   clnt_sd = accept(serv_sd, (struct sockaddr*)&clnt_adr, &clnt_adr_sz);

   while(1)
   {
      read_cnt = fread((void*)buf, 1, BUF_SIZE, fp);
      if (read_cnt < BUF_SIZE)
      {
         write(clnt_sd, buf, read_cnt);
         break;
      }
      write(clnt_sd, buf, BUF_SIZE);
   }

   shutdown(clnt_sd, SHUT_WR);
   read(clnt_sd, buf, BUF_SIZE);
   printf("Message from client: %s \n", buf);

   fclose(fp);
   close(clnt_sd); close(serv_sd);
   return 0;
}
```

write 종료, still read 가능

Example: file_client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define BUF_SIZE 30
void error_handling(char *message);

int main(int argc, char *argv[])
{
   int sd;
   FILE *fp;

   char buf[BUF_SIZE];
   int read_cnt;
   struct sockaddr_in serv_adr;
   if (argc != 3) {
      printf("Usage: %s <IP> <port>\n", argv[0]);
      exit(1);
   }
```

```
   fp = fopen("receive.dat", "wb");
   sd = socket(PF_INET, SOCK_STREAM, 0);

   memset(&serv_adr, 0, sizeof(serv_adr));
   serv_adr.sin_family = AF_INET;
   serv_adr.sin_addr.s_addr = inet_addr(argv[1]);
   serv_adr.sin_port = htons(atoi(argv[2]));

   connect(sd, (struct sockaddr*)&serv_adr, sizeof(serv_adr));

   while ((read_cnt = read(sd, buf, BUF_SIZE )) != 0)
      fwrite((void*)buf, 1, read_cnt, fp);

   puts("Received file data");
   write(sd, "Thank you", 10);
   fclose(fp);
   close(sd);
   return 0;
}
```

Questions #1-1

1. 소켓이란 무엇이며, 애플리케이션 계층과 전송 계층 간의 역할은 어떤 방식으로 분리되어 있는가?
2. socket() 함수 호출 시 지정하는 domain, type, protocol 각각의 의미는 무엇이며, TCP 소켓 생성을 위한 적절한 인자는 무엇인가?
3. TCP 서버가 클라이언트의 연결 요청을 받아들이기까지 필요한 함수 호출 흐름을 설명하라. (tcp.server.c & tcp_client.c 참고)
4. 클라이언트 측에서 서버와 연결을 맺기 위해 connect() 함수를 사용할 때 내부적으로 어떤 정보들이 전달되고 처리되는가?
5. read()가 return 하는 값은 무엇이며, tcp_client.c의 while 문에서 read()의 return 값에 따라 어떻게 동작하는가?

Questions #1-2

6. bind() 함수 호출 시 필요한 sockaddr_in 구조체의 각 필드는 어떤 정보를 담고 있으며, 왜 htons()와 htonl()이 필요한가?
7. inet_addr()와 inet_aton() 함수의 차이점은 무엇이며, 각각의 반환 방식이 프로그램 설계에 어떤 영향을 주는가?
8. inet_ntoa() 함수를 사용할 때 주의할 점과 이 함수가 반환하는 값의 유효 기간에 대해 설명하라.
9. gethostbyname()과 gethostbyaddr() 함수는 각각 어떤 상황에서 사용되며, 반환된 hostent 구조체의 정보를 어떻게 활용하는가?

Questions #1-3

10. TCP가 message boundary를 유지하지 않는다는 것은 어떤 의미이며, 이에 따라 프로그래머가 고려해야 할 점은 무엇인가?
11. Iterative 서버와 Concurrent 서버의 구조적 차이점을 설명하고, 각각의 장단점을 비교하라.
12. echo_server.c 코드에서 반복문을 통해 클라이언트 요청을 처리하는 방식의 한계와 개선 방안을 서술하라.
13. pkt_t 구조체를 활용하여 모드와 메시지를 함께 전달할 때, Binary 데이터 전송 시 주의해야 할 점은 무엇인가?
14. 구조체 기반 통신에서 read() 및 write() 함수 호출 시 데이터의 정렬(alignment)이나 패딩(padding) 문제가 발생할 수 있는 경우는 어떤 경우인가?

데이터보내고 받는 순서
byte align
TCP boundary 문제
(TCP 바운더리 없어서 일부만 read)

Questions #1-4

read 가 닫힘 (shutdown)- EOF 신호 전송, read == 0 - sd close (일종의 free)

15. TCP 소켓에서 입출력 버퍼(Input/Output Buffer)는 어떤 시점에 생성되며, 연결 종료 시 어떤 방식으로 처리되는가?
16. shutdown() 함수와 close() 함수의 차이점을 설명하고, Half-close가 필요한 시나리오를 제시하라.

half close

17. 서버가 SHUT_WR 옵션으로 연결을 종료한 이후, 클라이언트는 어떤 방식으로 남은 데이터를 수신해야 하는가?
18. file_server.c 예제에서 shutdown()을 사용한 후 read()를 수행하는 이유와 그 의미를 구체적으로 설명하라.

숙제 #1

- TCP 기반 파일 다운로드 프로그램 구현
  - 1. 클라이언트가 서버에 접속 (TCP 이용)
  - 2. 서버 프로그램이 실행 중인 디렉토리의 모든 파일 목록 (파일 이름, 파일 크기)을 클라이언트에게 전송
  - 3. 클라이언트는 서버가 보내 온 목록을 보고 파일 하나를 선택
  - 4. 서버는 클라이언트가 선택한 파일을 클라이언트에게 전송
  - 5. 전송된 파일은 클라이언트 프로그램이 실행 중인 디렉토리에 동일한 이름으로 저장됨.
  - 6. 2~5번 과정 반복

    - 사용자 Interface는 자유롭게 해도 됨. 단, 사용하기 쉽도록 메뉴나 명령어에 대한 설명 필요
    - 텍스트 파일 뿐만 아니라 바이너리 파일도 전송할 수 있어야 함